

Хранение клиентских данных

Данные, хранимые в БД сервисов облака, делятся на **клиентские**, т.е. с привязкой к клиенту, и **неклиентские**. В этой статье рассмотрим, как эффективно организовать хранение клиентских данных.

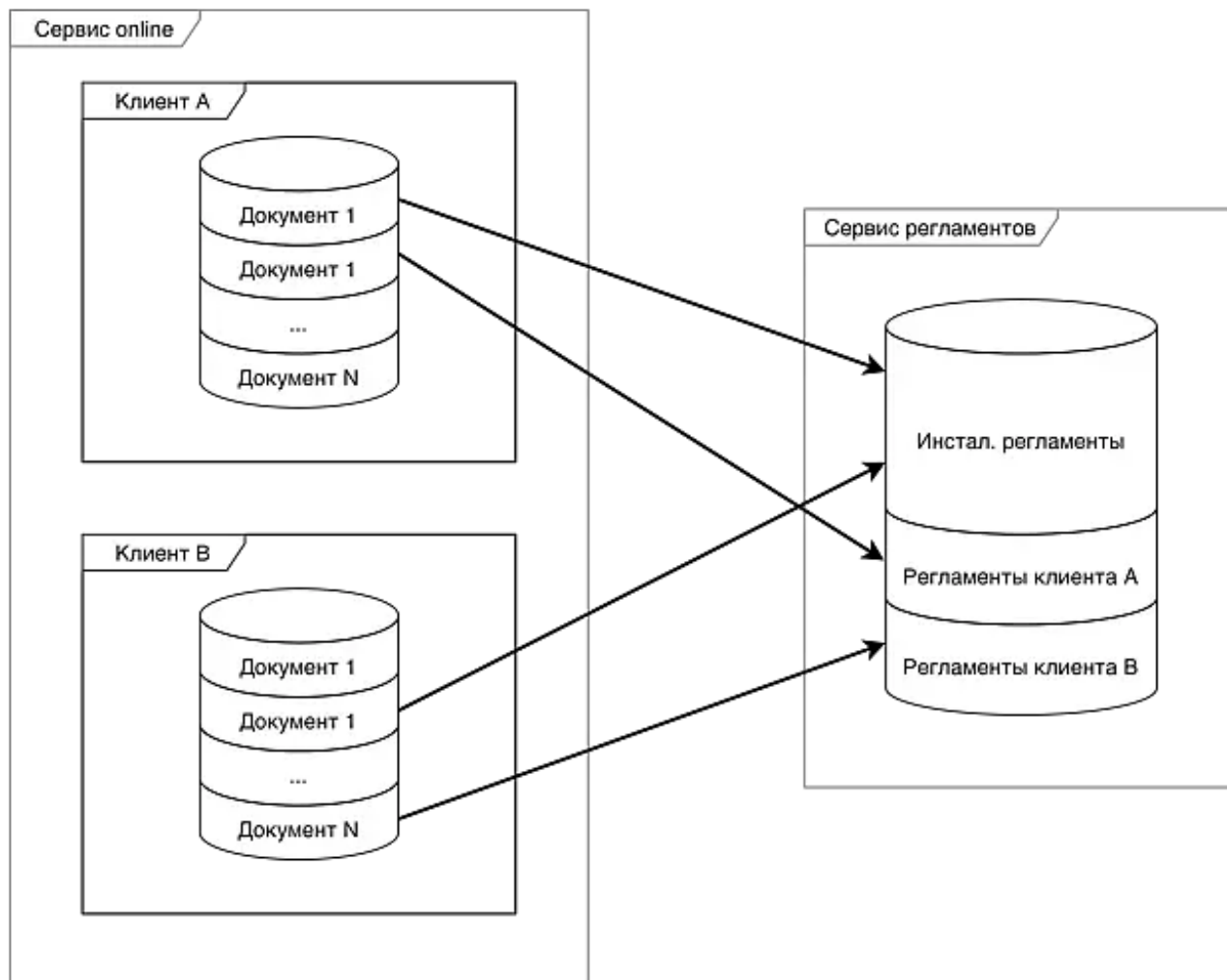


Рис. 1. Способы хранения клиентских данных на примере Документа и его Регламента

Платформа СБИС предоставляет два способа:

- в ячейке (схеме клиента) [multitenant-базы данных](#) сервиса [online.sbis.ru](#).

Целесообразно выбрать этот способ хранения данных, если:

- хранимые данные относятся к одному клиенту;
- к данным обращаются часто и результат нужно получить максимально быстро.

Преимущества этого способа заключаются в том, что, во-первых, не тратятся ресурсы на межсервисное взаимодействие, во-вторых, операции управления данными, такие как удаление клиента, клонирование схемы и перенос данных между аккаунтами, поддерживаются "из коробки".

Например, получение списка задач — частая операция. Запросом к основному сервису можно получить все данные, без потерь на межсервисное общение. Поэтому хранение "задач" организовано в таблицах основного multitenant-сервиса.

- в [прочем сервисе](#)

Этот способ следует использовать, если:

- данные относятся к нескольким клиентам, например, сервис сверки, в котором одни и те же сведения о счет-фактуре связаны как с продавцом, так и с покупателем.
- небольшое количество клиентов будет пользоваться функционалом: неэффективно создавать таблицы во всех multitenant-схемах, т.к. у большинства клиентов они будут пустые, а ресурсы на таблицу и индекс тратятся.

Например, регламенты хранятся в отдельном сервисе, т.к. большинство клиентов пользуются одними и теми же инсталляционными регламентами. Клиентов, которые создают свои регламенты, существенно меньше, поэтому эффективнее их хранить вместе с инсталляционными регламентами, но с привязкой к конкретному клиенту.



Важно!

Поддержку событий [удаления клиента](#), клонирования схемы и перенос данных между аккаунтами в прочем сервисе обеспечивает разработчик сервиса.

Пример подписки на событие удаления клиента.

```
1 sub = event.Subscription()
2 sub.channel = 'cloud-ctrl.clientdeleting'
3 ...
4 sub.SetCallback(
5     'Sync.DeleteClientContent',
6     event._MsgBody_
7 )
```

Пример метода удаления клиентских данных.

```
1 def delete_client_content(msg: sbis.Record) -> bool:
2     """
3     Удаляет контент, созданный клиентом на сервисе theming, в ответ на
4     событие cloud-ctrl.clientdeleting
5     Возвращает:
6     True - если был найден и удален контент, которым владеет клиент.
7     False - в ином случае.
8     """
9     client_id = msg.Get('@Пользователь')
10    if client_id:
11        <код удаления клиентских данных>
12        return True
13    return False
```